

DATA**ACTIVIST**

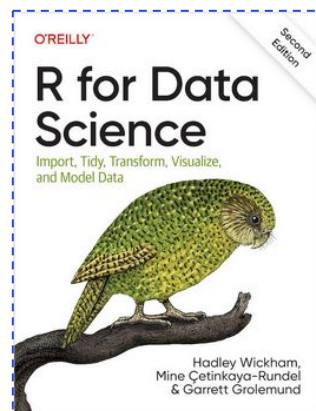
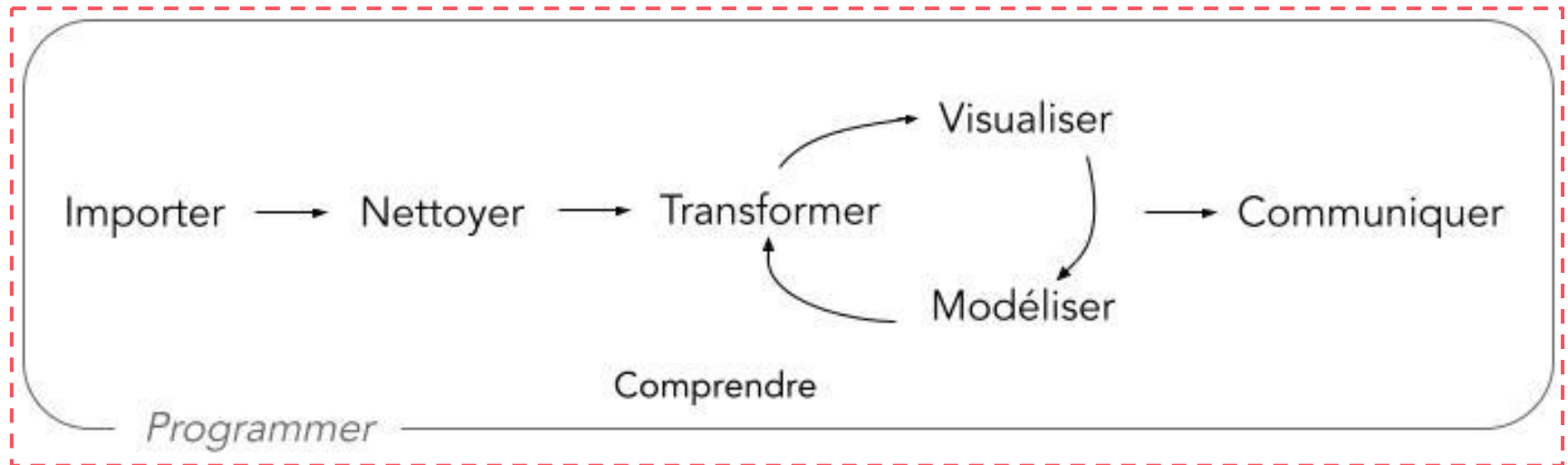
Communauté
d'**Agglomération** de
La Rochelle



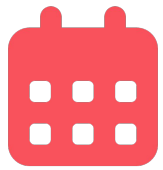
Introduction au langage de programmation R

25 juin 2026

Le programme de cette journée ; le schéma d'exploitation de la donnée

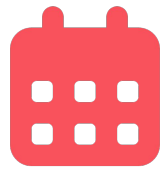


Source : [R for Data Science \(2e\)](#)



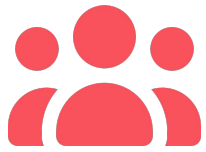
PROGRAMME MATINÉE

- **Introduction : la programmation, R et RStudio**, *comprendre les principes de la programmation, et les spécificités du langage R et de l'interface RStudio ;*
- **Importer des données ;**
- **Nettoyer et transformer des données :** *connaître les principales fonctions pour manipuler les données avec des exercices de mise en pratique ;*



PROGRAMME APRÈS-MIDI

- **Datavisualisation : comment faire de la visualisation sous R avec ggplot,** *appréhender la librairie de visualisation de données du tidyverse, avec des exercices de mise en pratique ;*
- **Cartographie : comment faire des cartes interactives en R,** *aperçu de la librairie leaflet ;*
- **Exercice final :** *reproduire une datavisualisation impliquant le traitement et la visualisation de données ;*
- **Conclusion de la journée :** vos retours, le mot de fin.



Avant de commencer

Faisons un rapide tour de table :

- qui êtes-vous ? 🔍
- sur une échelle de 1 à 3, à combien estimez-vous votre **niveau** en manipulation de données ? ★★☆☆
- sur une échelle de 1 à 3, à combien estimez-vous votre **intérêt** pour la manipulation de données ? ★★☆☆
- qu'attendez-vous de cette journée ? 💡

La programmation, R et RStudio

```
    }).done(function(response) {  
      for (var i = 0; i < response.length; i++) {  
        var layer = L.marker(  
          [response[i].latitude, response[i].longitude]  
          // , {icon: myIcon}  
        );  
        layer.addTo(group);  
  
        layer.bindPopup(  
          "<p>" + "Species: " + response[i].species + "<br>" +  
          "<p>" + "Description: " + response[i].description + "<br>" +  
          "<p>" + "Seen at: " + response[i].latitude + " " + response[i].longitude + "<br>" +  
          "<p>" + "On: " + response[i].sighted_at + "</p>"  
        );  
      }  
  
      $('select').change(function() {  
        species = this.value;  
      });  
    });  
  }  
  $.ajax({  
    url: queryURL,  
    method: "GET"  
  }).done(function(response) {  
    for (var i = 0; i < response.length; i++) {  
      var layer = L.marker(  
        [response[i].latitude, response[i].longitude]  
        // , {icon: myIcon}  
      );  
      layer.addTo(group);  
    }  
  });  
}
```



Quelques ressources à présenter

➤ les fiches tuto de POSIT



CHEATSHEET |

Data tidying with tidyr
cheatsheet

[https://posit.co/
resources/chea
tsheets/](https://posit.co/resources/cheatsheets/)

➤ un support complet pour approfondir les notions



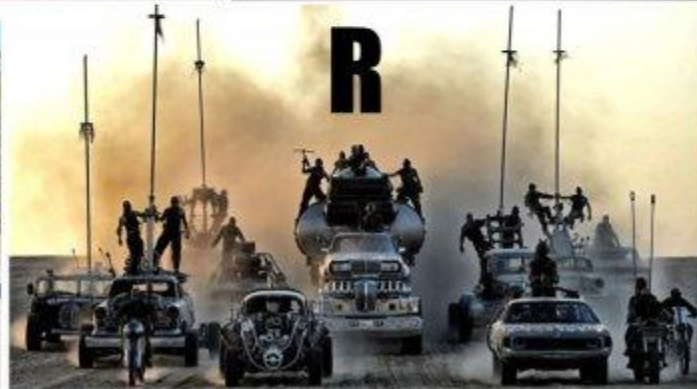
[https://dianethy.
github.io/cours
R/Introduction_R
.html](https://dianethy.github.io/cours_R/Introduction_R.html)

La **programmation** , tout un concept :

- **Définition** : ensemble des activités liées à la conception, l'écriture, le test et la maintenance de programmes informatiques à l'aide de langages de programmation (source : [wikipédia](https://fr.wikipedia.org/wiki/Programmation)).
- **Synonymes** : codage, développement.
- **Utilité** :
 - **autonomie** : pas de dépendance à un logiciel, permet d'automatiser des tâches répétitives
 - **scalabilité** : ce qui marche sur 100 lignes de données marche sur 100.000
 - **collaboration** : le code se partage facilement
 - **reproductibilité** : possible de refaire une analyse à l'identique
- **Usage** : de nombreux langages existent (HTML, Javascript, Java, C, Python, R...)

R, un langage de programmation **destiné** **aux statistiques**

If statistics programs/languages were cars...



Gratuit, puissant, design, avec une communauté importante ([Stack Overflow](#), [R-bloggers](#) etc.)

Les concepts clés :

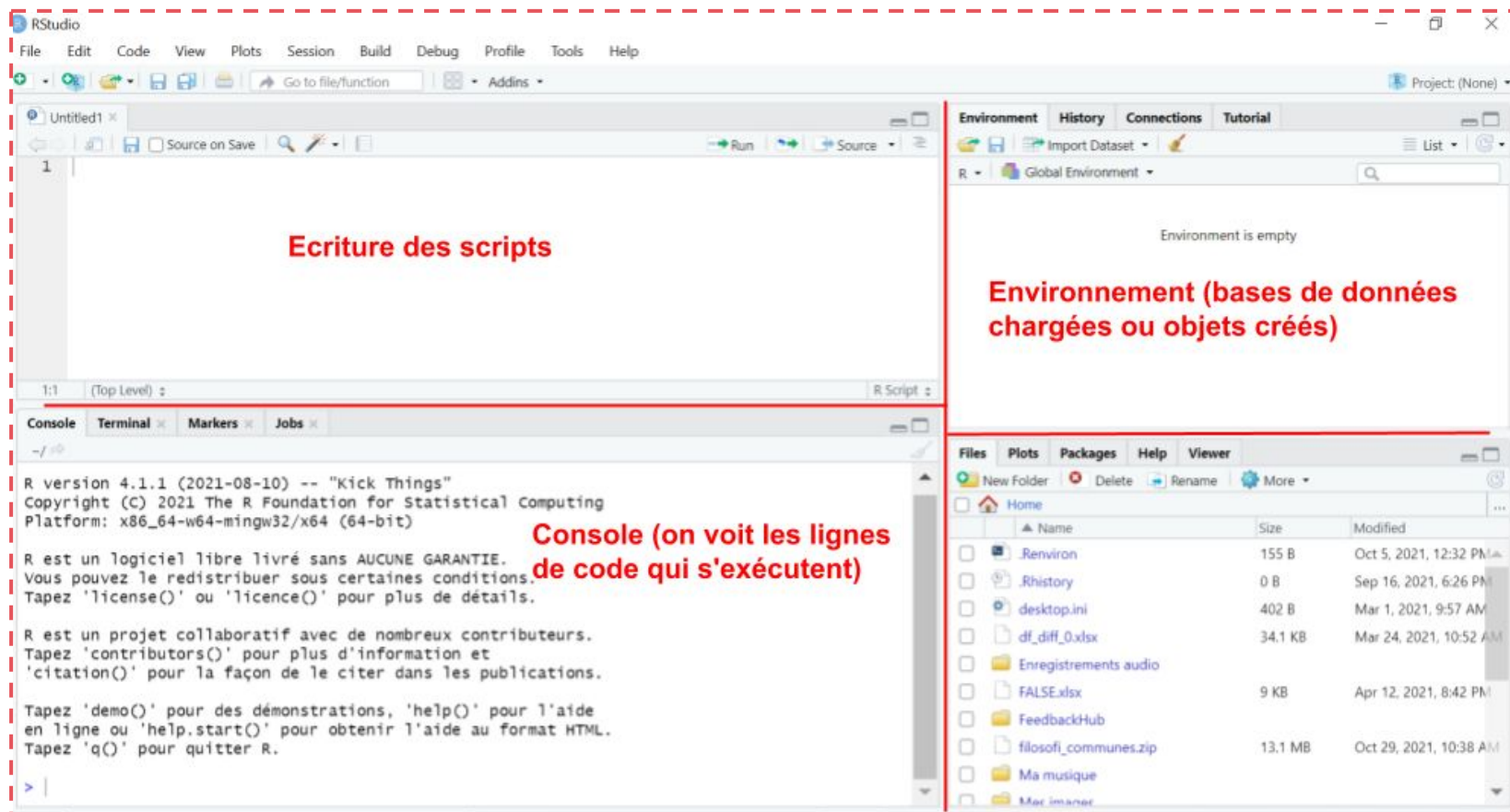
漢

- run
- package
- library
- script

Abc

- **exécuter** (une ligne de code)
- **ensemble de fonctions**
- **endroit où sont gardés les packages**
- **document où l'on code** (extension *.R*)

RStudio, une interface permettant d'interagir avec R



Assez parlé, ouvrons RStudio !

- Installez les packages dont nous aurons besoin aujourd'hui
 - `install.packages(c("readxl", "tidyverse", "janitor", "summarytools", "leaflet"))`
- Créez un nouveau script via: *File > New File > R script*
- Commencez à interagir par de simples opérations arithmétiques: $2+3$
- Exécutez les lignes de code: *Ctrl/Enter* ou bouton  Run
- Assignez des valeurs à des objets grâce à l'opérateur d'assignation `<-`
 - `x <- 2`
 - `y <- 7`
 - `resultat <- x * y`
 - `resultat`
 - `z <- "hello world"`
- Commentez le code via `#`

Deux manières de coder en R

- **R-base**

- exemple 1 : `exemple[,2]`
- exemple 2 : `f(g(h(exemple)))`

- **Tidyverse**

- exemple 1 : `exemple |> select(2)`
- exemple 2 : `exemple |> h() |> g() |> f()`



Le tidyverse fonctionne grâce au pipe (`%>%` ou `|>`), qui permet d'appliquer des fonctions à une base de données. L'opération à droite du pipe est appliquée à la valeur située à gauche du pipe, il devient alors possible d'enchaîner les traitements.

Récapitulatif des raccourcis clavier

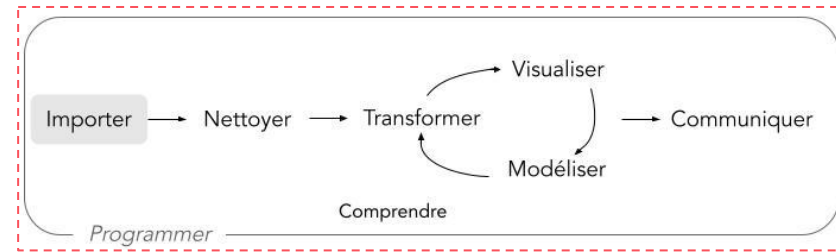
- pipe (%>% ou |>) : Ctrl, Maj, M
- opérateur d'assignation (<-) : Alt, -
- runner / exécuter une ligne : Ctrl, Enter
- ouvrir un nouveau script : Ctrl, Maj, N

Le tidyverse



Codons ensemble avec les principales fonctions du tidyverse

Importer des données



- **Excel**

- `library(readxl)`
- `exemple <- read_excel(path = "fichier.xlsx", sheet = "nom_feuille")`

- **CSV : valeurs séparées par des virgules**

- `library(readr)`
- `exemple <- read_csv("fichier.csv")`

- **SCSV : valeurs séparées par des points-virgules ("faux" CSV)**

- `library(readr)`
- `exemple <- read_delim("fichier.csv", delim = ";")`

💡 **ASTUCE :** si vous avez un doute, vous pouvez également importer les données en "press-bouton" via le Menu > File > Import Dataset >...

- **À vous !** Importer le jeu de données '**Prénoms donnés par département - Millésimé - France**' disponible sur OpenDataSoft :

- au format CSV en utilisant le lien [seulement en 1900 !!]
- si vous avez fini, au format tableur Excel en exportant le fichier [seulement en 1900 !!]

Importer des données

Solution

- Importer le jeu de données '**Prénoms donnés par département - Millésimé - France**' disponible sur OpenDataSoft :
 - en CSV en utilisant le lien

```
library(tidyverse)
data <-
read_delim("https://public.opendatasoft.com/api/explore/v2.1/catalog/datasets/demographyref-france-prenoms-departement-millesime/exports/csv?lang=fr&refine=birth_year%3A%221900%22&facet=facet(name%3D%22birth_year%22%2C%20disjunctive%3Dtrue)&timezone=Europe%2FBerlin&use_labels=true&delimiter=%3B",
, delim = ";")
```

- en excel en exportant le fichier

```
library(readxl)
data <-
read_excel("~/Downloads/demographyref-france-prenoms-departement-millesime.xlsx")
```



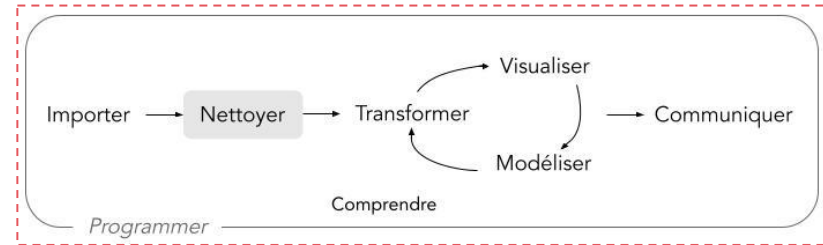
Astuce

Importer des données volumineuses

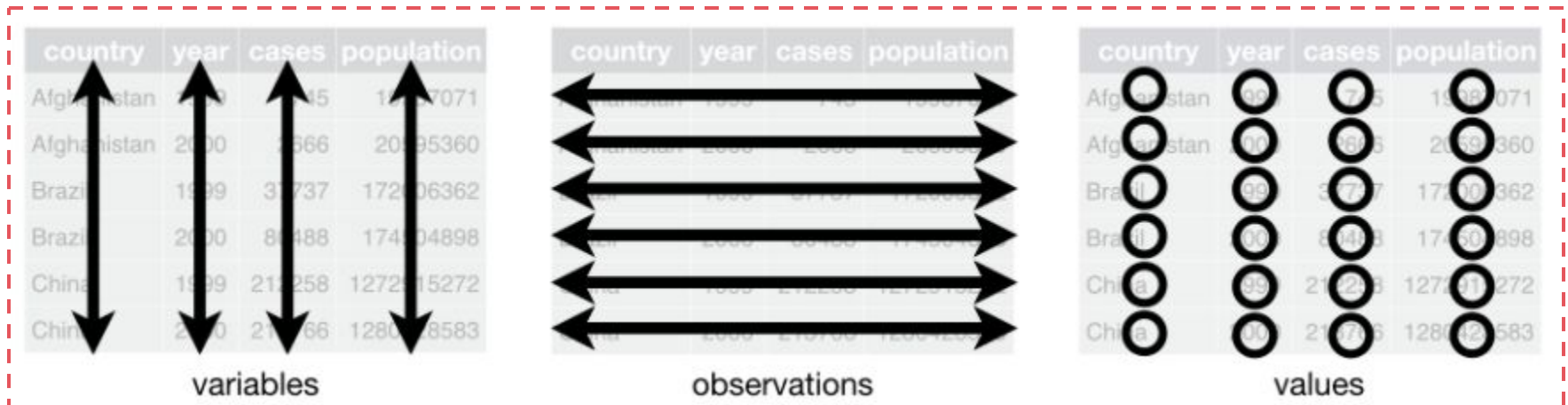
- La fonction `fread()` du package `data.table` permet d'importer des données volumineuses (>1GB) plus rapidement que les fonctions `readr`. Elle permet d'importer des formats CSV ou TSV, et on peut y spécifier les colonnes à importer pour limiter le volume des données.
 - `data.table::fread("dossier/mon_fichier_lourd.csv",
select = c("col1", "col4"))`
- Le package `data.table` contient de nombreuses fonctions pour manipuler de larges bases de données. La sémantique est cependant différente de celle que nous allons étudier aujourd'hui.

👉 [Lire la documentation complète de data.table](#)

Nettoyer des données



- **Structure de données propres**



- chaque ligne est une observation (entrée, enregistrement)
- chaque colonne est une variable
- une seule valeur dans chaque cellule

Nettoyer des données

- À vous !

- Quel jeu de données ci-dessous est proprement structuré ?
- Qu'est-ce qui ne va pas dans les 2 autres jeux ?

country	year	rate
Afghanistan	1999	745/19987071
Afghanistan	2000	2666/20595360
Brazil	1999	37737/172006362
Brazil	2000	80488/174504898
China	1999	212258/1272915272
China	2000	213766/1280428583

A.

country	year	key	value
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362
Brazil	2000	cases	80488
Brazil	2000	population	174504898
China	1999	cases	212258
China	1999	population	1272915272
China	2000	cases	213766
China	2000	population	1280428583

B.

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

C.

Nettoyer des données

Solution

- Quel jeu de données ci-dessous est proprement structuré ?
 - Le C.
- Qu'est-ce qui ne va pas dans les 2 autres jeux ?
 - Dans le A. on trouve 2 valeurs dans 1 case
 - Dans le B. les valeurs '*cases*' et '*population*' ne sont pas dans 2 colonnes séparées

Nettoyer des données

Avant toute analyse, il est nécessaire d'observer les données pour détecter les anomalies et les corriger

- **Types de données**

Famille de données	Structure	Nom retourné sous R	Exemple
<i>quantitatif</i>	entier	int = integer	3
	décimal inexact	dbl = double	3.4
	décimal exact	num = numeric	3.400001
<i>qualitatif</i>	chaîne de caractères	chr = character	Ville de Paris
	facteur à n niveaux	Factor	petit / moyen / grand
	facteur à 2 niveaux dit "booléen"	bool = boolean	0 / 1, True / False
<i>autre</i>	date	POSIX	13-12-1998

- **Pour les modifier**

```
as.integer()  
as.double()  
as.numeric()  
as.character()  
as.factor()  
as.logical()  
as.POSIXct()
```

- **Exemple de modification d'un typage de variable**

Si je veux mettre ma variable au format numérique : `as.numeric(ma_variable)`

Nettoyer des données

Avant toute analyse, il est nécessaire d'observer les données pour détecter les anomalies et les corriger

- **Les fonctions pour observer les données**
 - `glimpse(data)` : structure des données (on y voit tous les types)
 - `names(data)` : nom des colonnes
 - `view(dfSummary(data))` : tableau résumé des données
- **À vous !** Répondre aux questions suivantes :
 - le typage de la variable '*Code Officiel Région*' est-il correct ?
 - quel est le nombre de naissances maximal pour un prénom ?

Nettoyer des données

Solution

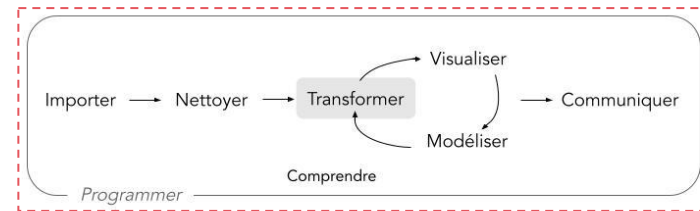
- le typage de la variable '*Code Officiel Région*' est-il correct ?

Le code officiel de la région correspond à un identifiant et non à un nombre à proprement parler (somme, taux, note etc.), donc doit être considéré comme une chaîne de caractères, qui correspond au format '*character*' sous R. Avec la fonction **glimpse(data)** on voit qu'il est considéré comme une chaîne de caractères ('*chr*'), donc son typage est **correct**.

- quel est le nombre de naissances maximal pour un prénom ?

La fonction **view(dfSummary(data))** nous informe que le maximum est de **2519**.

Transformer des données



Les 7 principales fonctions du **dplyr**, package de transformation de données contenu dans le **tidyverse**, sont les suivantes :

- **rename()** : pour renommer des colonnes
- **select()** et **relocate()** : pour sélectionner, supprimer ou changer l'ordre des colonnes
- **filter()** : pour sélectionner certaines lignes selon une condition
- **mutate()** : pour créer de nouvelles variables
- **summarise()** : pour résumer plusieurs valeurs en 1 valeur
- **group_by()** : pour grouper les observations avant d'appliquer une fonction

💡 **ASTUCE** : vous devrez maintenant utiliser le "pipe" (`|>` ou `|>`) pour appliquer ces fonctions. Vous pouvez l'écrire sur votre script grâce au raccourci clavier : Ctrl, Maj, M

Transformer des données : `rename()`

Cette fonction permet de renommer une ou plusieurs colonnes d'une base de données

- **Paramètres :**

- `names(data)` pour connaître le nom des colonnes des données
- `data <- data |> rename(nouveau_nom = `ancien nom`,
nv_nom = ancien_nom)`

- **Exemple :**

- `exemple <- exemple |> rename(tx_chomage = `Taux de chômage`,
regions = régions)`

- **À vous !**

- renommer la colonne 'Année de naissance' par 'annee_naissance'
- renommer la colonne 'Code Officiel Département' par 'code_departement'
- renommer la colonne 'Nom Officiel Département' par 'nom_departement'
- renommer la colonne 'Nombre de naissances' par 'nb_naissances'

Transformer des données : `rename()`

Solution

- renommer la colonne 'Année de naissance' par 'annee_naissance'
- renommer la colonne 'Code Officiel Département' par 'code_departement'
- renommer la colonne 'Nom Officiel Département' par 'nom_departement'
- renommer la colonne 'Nombre de naissances' par 'nb_naissances'

```
data <- data |>
  rename(annee_naissance = `Année de naissance`,
         code_departement = `Code Officiel Département`,
         nom_departement = `Nom Officiel Département`,
         nb_naissances = `Nombre de naissances`)
```



Astuce

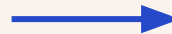
Renommer toutes les colonnes en une ligne de code

- La fonction `clean_names()` du package `janitor` applique automatiquement des règles pour transformer les noms de colonnes en `snake_case` en une seule opération

- `data |> janitor::clean_names()`

- Exemple :

Mon id	une_LETTRE	évènements
1	A	G
2	B	H



mon_id	une_lettre	evenements
1	A	G
2	B	H

Avant

Après



[Lire la documentation complète de janitor](#)

Transformer des données : `select()` et `relocate()`

La fonction `select()` permet de **sélectionner** ou **supprimer** certaines colonnes, par leur nom ou leur position dans la base de données.

La fonction `relocate()` permet de **changer l'ordre** des colonnes, par leur nom ou leur position dans la base de données.

- **Paramètres :**

- `data <- data |> select(col1, col3, col2, col4)`

- **Exemples :**

- `exemple <- exemple |> select(!col2)`

- `exemple <- exemple |> select(1:3, 7, 4:6)`

- **À vous !**

- **créer un nouvel objet** sans les variables *'annee_naissance'* et *'Année triable'*
- **créer un nouvel objet** avec toutes les variables de *'Sexe'* à *'nb_naissances'*
- **créer un nouvel objet** où *'code_departement'* est la première variable, en gardant les autres colonnes telles quelles

Transformer des données : `select()`

Solution

- **créer un nouvel objet** sans les variables *'annee_naissance'* et *'Année triable'*

```
dat_select1 <- data |> select(!annee_naissance, !`Année triable`)  
dat_select1 <- data |> select(!c(annee_naissance, `Année triable`))
```

- **créer un nouvel objet** avec toutes les variables de *'Sexe'* à *'nb_naissances'*

```
dat_select2 <- data |> select(Sexe:nb_naissances)
```

- **créer un nouvel objet** où *'code_departement'* est la première variable, en gardant les autres colonnes telles quelles

```
dat_relocate1 <- data |> relocate(code_departement)
```

Transformer des données : `filter()`

Cette fonction sert à sélectionner des lignes dans une base de données, répondant à certains critères ou conditions logiques

- **Paramètres :**

- `data <- data |> filter(col1 == "valeur")`
- opérations possibles : `==`, `!=`, `>`, `<`, `>=`, `<=`
- combinaisons d'opérations : `&` pour "et", `|` pour "ou"
- utiliser `nrow()` pour compter le nombre de ligne filtrées

- **Exemples :**

- `exemple <- exemple |> filter(col1 != 10 & col3 == "red")`
- `exemple |> filter(col > 0 | col1 <= 100) |> nrow()`

- **À vous !**

- **dans un nouvel objet**, filtrer les prénoms féminins
- **dans un nouvel objet**, filtrer les départements Aisne et Gironde

Transformer des données : `filter()`

Solution

- dans un nouvel objet, filtrer les prénoms féminins

```
dat_filter1 <- data |> filter(Sexe == "Féminin")
```

- dans un nouvel objet, filtrer les départements Aisne et Gironde

```
dat_filter2 <- data |> filter(nom_departement == "Aisne" |  
                             nom_departement == "Gironde")
```


Transformer des données : `mutate()`

Cette fonction permet de créer de nouvelles variables à partir des variables existantes, elle est très utile pour l'analyse de données

- **Paramètres :**

- `data <- data |> mutate(new_col = col1 * col2)`

- **Exemples :**

- `exemple <- exemple |> mutate(new_col = col1 / col2 * 100)`

- `exemple <- exemple |> mutate(new_col2 = col3 - 52)`

- `exemple <- exemple |> mutate(new_col2 = "Un nom à donner")`

- **À vous !**

- créer une **nouvelle variable** du nombre de naissances * 10

- créer une **nouvelle variable** de la part de chaque naissance (chaque ligne) sur toutes les naissances de 1900

Transformer des données : `mutate()`

Solution

- créer une **nouvelle variable** du nombre de naissances * 10

```
data <- data |> mutate(nb_naissances10 = nb_naissances * 10)
```

- créer une **nouvelle variable** de la part de chaque naissance (chaque ligne) sur toutes les naissances de 1900

```
data <- data |> mutate(taux = nb_naissances /  
sum(nb_naissances) *100)
```

Transformer des données : `summarise()`

Cette fonction permet de résumer plusieurs valeurs en une seule, ce qui est très utile pour avoir quelques statistiques sur les bases de données

- **Paramètres :**

- `data |> summarise(moyenne = mean(col1))`
- statistiques possibles : `mean`, `median`, `min`, `max`, `range`, `sum`, `n`
- si au moins une valeur manque (`NA`), ajouter l'argument `na.rm = TRUE`

- **Exemple :**

- `exemple |> summarise(min = min(col1, na.rm = TRUE),
max = max(col1, na.rm = TRUE))`

- **À vous !**

- calculer le nombre de naissances moyen et médian
- calculer l'étendue du nombre de naissances

Transformer des données : `summarise()`

Solution

- calculer le nombre de naissances moyen et médian

```
data |> summarise(moy = mean(nb_naissances),  
                  med = median(nb_naissances))
```

- calculer l'étendue du nombre de naissances

```
data |> summarise(etendue = range(nb_naissances))
```

Transformer des données : `group_by()`

Cette fonction permet de grouper les variables, elle est spécialement utile en analyse de données pour calculer des statistiques par groupe. Depuis janvier 2023, l'argument `.by` peut-être ajouté directement **dans** les fonctions du `dyplr`.

- **Paramètres :**

- ancienne manière de coder : `data |> group_by(group) |> summarise(min = min(col)) |> ungroup()`
- nouvelle manière de coder :
`data |> summarise(min = min(col), .by = group)`
- combiner avec les opérations déjà présentées : `mean`, `median`, `min`, etc

- **Exemple :**

- `exemple |> mutate(n = n(), .by = group_col)`

- **À vous !**

- calculer le nombre de naissances par sexe **dans un nouvel objet**
- calculer **dans une nouvelle variable de la base**, le nombre de prénoms (lignes) par région

Transformer des données : `group_by()`

Solution

- calculer le nombre de naissances par sexe **dans un nouvel objet**

```
group_data <- data |> summarise(naissances_sexe =  
sum(nb_naissances), .by = Sexe)
```

- calculer **dans une nouvelle variable de la base**, le nombre de prénoms (lignes) par région

```
data |> mutate(n_reg = n(), .by = `Nom Officiel Région`)
```

Une mise en pratique avant d'aller déjeuner



STATIONNEMENT – PLACE EN ZONE BLEUE

Exercice de mise en pratique

Vous avez 15 minutes pour effectuer ces quelques manipulations sur un jeu de données

- Depuis la base de données "**Stationnement – Place en zone bleue**" disponible sur le portail open data de La Rochelle :
 - quel est l'id de tronçon (pzb_troncon_id) du pzb_id '75' de la voie "Avenue des Corsaires" ?
 - combien y a-t-il de places de parking (lignes) sur le boulevard ANDRE SAUTEL ?

Exercice de mise en pratique

Solution

- Depuis la base de données "**Stationnement – Place en zone bleue**"

disponible sur le portail open data de La Rochelle :

- quel est l'id de tronçon (pzb_troncon_id) du pzb_id '75' de la voie "Avenue des Corsaires" ?

```
place_bleue <-  
read_csv("https://opendata.agglo-larochelle.fr/d4c/api/records/2.0/downloadfile/format=csv&resource_id=407dd9c7-5f0c-4f69-8746-ba2e1caaa291&use_labels_for_header=true&user_defined_fields=true")  
place_bleue |>  
  filter(pzb_id == 75 & pzb_voie == "Avenue DES CORSAIRES") |>  
  select(pzb_troncon_id)
```

- combien y a-t-il de places de parking (lignes) sur le boulevard ANDRE SAUTEL ?

```
place_bleue |> filter(pzb_voie == "Boulevard ANDRE SAUTEL") |> nrow()
```

Conclusion de la matinée

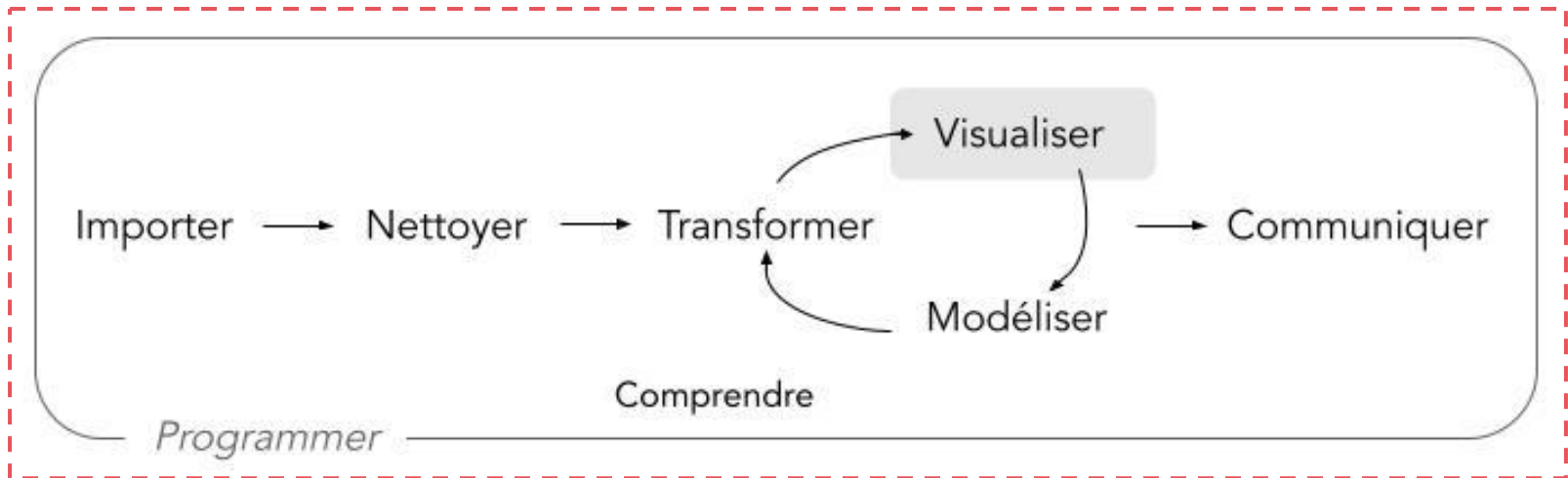


Avez-vous des questions ?

Résumons cette matinée

- Vous comprenez à quoi sert la programmation et pourquoi utiliser R via l'interface RStudio
- Vous avez fait vos premiers pas en code R :
 - import de données
 - nettoyage de données
 - transformation de données (`rename()`, `select()`, `relocate()`, `filter()`, `mutate()`, `summarise()`, `group_by()`)
- Vous avez mis en pratique toutes ces fonctions au fur et à mesure

Au programme de cette après-midi



```
print("Bon appétit")  
[1] "Bon appétit"
```